

PERN Full-Stack Technical Case Study

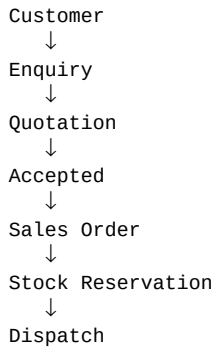
Industrial ERP Management System — Complete Project Guide

Stack: PostgreSQL + Express.js + React.js + Node.js

Goal: Build a small ERP application for Customer Enquiry → Quotation → Sales Order → Inventory Reservation → Dispatch.

Deadline note: The HR notice says 24 hours, while the detailed assignment says 48 hours. Follow the shorter 24-hour deadline unless HR/college confirms otherwise.

1. Project Overview



A customer requests products. Sales User creates the enquiry and quotation. After acceptance, it becomes a Sales Order. Admin confirms the order, reserves stock, and finally dispatches the products.

2. Roles & Permissions

Feature	Sales User	Admin
Login	Yes	Yes
Create customer/enquiry	Yes	No
View enquiries	Yes	Yes
Create quotation	Yes	No
Accept/Reject quotation	Yes	No
Convert quotation to order	Yes	No
View inventory	Yes	Yes
Confirm order	No	Yes
Reserve stock	No	Yes
Dispatch	No	Yes

3. Main Screens

- Login
- Enquiries — create/view enquiries
- Quotations — create, calculate, accept/reject
- Sales Orders — view, stock availability, confirm/reserve, dispatch

Keep the UI clean, functional and responsive. Do not spend excessive time on visual design.

4. Database Design

users
customers
products
inventory
enquiries
enquiry_items
quotations
quotation_items
sales_orders
sales_order_items
dispatches
dispatch_items

Customer 1 ■■■■ * Enquiries
Enquiry 1 ■■■■ * Enquiry Items ■■■■ Product
Enquiry 1 ■■■■ 1 Quotation
Quotation 1 ■■■■ * Quotation Items ■■■■ Product
Quotation 1 ■■■■ 0/1 Sales Order
Sales Order 1 ■■■■ * Sales Order Items ■■■■ Product
Product 1 ■■■■ 1 Inventory
Sales Order 1 ■■■■ * Dispatches

5. Table Structure

users

id, name, email UNIQUE, password_hash, role, created_at

customers

id, company_name, contact_person, mobile, email, city, created_at

products

id, product_code UNIQUE, product_name, category, unit, base_price

inventory

id, product_id FK UNIQUE, physical_quantity, reserved_quantity, updated_at

enquiries

id, enquiry_number UNIQUE, customer_id FK, enquiry_date, required_date, notes, status

enquiry_items

id, enquiry_id FK, product_id FK, quantity

quotations

id, quotation_number UNIQUE, enquiry_id FK UNIQUE, customer_id FK, valid_until, status, total_amount

quotation_items

id, quotation_id FK, product_id FK, quantity, unit_price, discount_percent, gst_percent, line_amount

sales_orders

id, order_number UNIQUE, quotation_id FK UNIQUE, customer_id FK, order_date, total_amount, status

sales_order_items

id, sales_order_id FK, product_id FK, quantity

dispatches

id, dispatch_number UNIQUE, sales_order_id FK, dispatch_date, vehicle_number, driver_name

dispatch_items

id, dispatch_id FK, product_id FK, quantity

6. Product Seed Data

Code	Product	Category	Unit	Price
IND-001	Industrial Pump	Pump	Piece	■5,000
IND-002	Hydraulic Valve	Valve	Piece	■2,500
IND-003	Steel Bearing	Bearing	Piece	■1,200
IND-004	Pressure Gauge	Instrument	Piece	■800
IND-005	Conveyor Belt	Conveyor	Meter	■1,500
IND-006	Electric Motor	Motor	Piece	■8,000

7. Enquiry

ENQ-001
Customer: ABC Engineering Pvt. Ltd.
Enquiry Date: 17-09-2026
Required Date: 25-09-2026

Industrial Pump 100
Hydraulic Valve 40

Status: NEW

One enquiry can contain multiple products, so use enquiry_items.

8. Quotation

Quotation is created against an enquiry. Backend must calculate/validate the total; React must not be trusted with a final amount.

Quantity = 100
Unit Price = ■5,000
Base = ■5,00,000

10% Discount = ■50,000
After discount = ■4,50,000

18% GST = ■81,000

Final = ■5,31,000

Statuses: DRAFT → SENT → ACCEPTED, or SENT → REJECTED.

9. Sales Order

Only an ACCEPTED quotation can create a Sales Order. DRAFT and REJECTED quotations must fail. Use UNIQUE(quotation_id) in sales_orders to prevent duplicate orders.

10. Inventory

Physical = 100
Reserved = 30
Available = 70

Reserve 60:

Physical = 100
Reserved = 90
Available = 10

Reservation does not reduce physical stock.

11. Simultaneous Reservation

This is a key backend challenge.

Available = 100

Request A → 80

Request B → 50

Both must NOT succeed.

Use PostgreSQL transaction + row-level lock:

```
BEGIN
  SELECT inventory ... FOR UPDATE
  check available
  update reserved_quantity
COMMIT
```

The lock makes requests serialize on the same inventory row, preventing over-reservation.

12. Dispatch

Before:
Physical = 100
Reserved = 60
Available = 40

Dispatch = 60

After:
Physical = 40
Reserved = 0
Available = 40

- Dispatch quantity cannot exceed reserved quantity.
- Duplicate dispatch must be prevented.
- Cancelled orders cannot be dispatched.
- Use a transaction for inventory + dispatch updates.

13. Authentication & RBAC

```
Login
↓
bcrypt password verification
↓
JWT generated
↓
React sends token
↓
Protected API
↓
JWT verification
↓
Role check
↓
Controller
```

Backend authorization is mandatory. Hiding buttons in React is not enough.

14. REST APIs

```
POST  /api/auth/login

POST  /api/customers
GET   /api/customers

GET   /api/products
GET   /api/inventory

POST  /api/enquiries
GET   /api/enquiries
GET   /api/enquiries/:id

POST  /api/quotations
GET   /api/quotations
GET   /api/quotations/:id
PATCH /api/quotations/:id/status
POST  /api/quotations/:id/convert

GET   /api/sales-orders
GET   /api/sales-orders/:id
POST  /api/sales-orders/:id/confirm
POST  /api/sales-orders/:id/dispatch
```

15. Backend Folder Structure

```
backend/
  src/
    config/database.js
```

```
■   ■■■ models/
■   ■■■ controllers/
■   ■■■ routes/
■   ■■■ middleware/
■   ■   ■■■ auth.js
■   ■   ■■■ role.js
■   ■■■ services/
■   ■■■ validators/
■   ■■■ server.js
■■■ tests/
■■■ .env
■■■ package.json
■■■ README.md
```

16. Frontend Folder Structure

```
frontend/
■■■ src/
■   ■■■ components/
■   ■■■ pages/
■   ■   ■■■ Login.jsx
■   ■   ■■■ Enquiries.jsx
■   ■   ■■■ Quotations.jsx
■   ■   ■■■ SalesOrders.jsx
■   ■■■ services/
■   ■■■ context/
■   ■■■ App.jsx
■■■ .env
■■■ package.json
```

17. Mandatory Automated Tests

- Quotation total is calculated correctly.
- DRAFT/REJECTED quotation cannot create a Sales Order.
- Same quotation cannot generate duplicate Sales Orders.
- Cannot reserve more inventory than available.
- Unauthorized user cannot perform an Admin-only operation.

Bonus: send two simultaneous reservation requests and verify both cannot succeed when combined quantity exceeds available stock.

18. 24-Hour Build Plan

Time	Work
1 hr	Setup: Node, Express, PostgreSQL, Sequelize, JWT, bcrypt, React
2 hrs	Models, relationships, migrations, seed
2 hrs	Login, JWT, RBAC
3 hrs	Customer, enquiry, quotation, calculations
2 hrs	Sales Order + duplicate prevention
2 hrs	Inventory transaction/locking + dispatch
2 hrs	React screens + API integration
1 hr	Automated tests
1 hr	README, ER diagram, API docs, demo recording

19. What Not to Spend Time On

- Fancy dashboard
- Animations
- Complex Redux architecture
- Dark/light theme
- Charts
- Extra notifications
- Unrequested features

Focus on backend correctness, database relationships, validation, transactions, RBAC and tests.

20. Submission Checklist

- Complete Git repository with reasonable development history
- README: tech stack, setup, DB setup, environment variables, migrations/seed, run commands, tests, login credentials
- Database Schema / ER Diagram
- Postman / Swagger / equivalent API documentation
- Maximum 5-minute demo video
- Submit details and case study through the Google Form

21. Demo Recording Flow

```
Login as Sales User
→ Create Customer/Enquiry
→ Create Quotation
→ Accept
→ Convert to Sales Order
→ Logout
→ Login as Admin
→ Confirm & Reserve
→ Show updated stock
→ Dispatch
→ Show final stock
```

Suggested narration:

First, I am logging in as a Sales User. I am creating a customer enquiry with the required product and quantity. Now I am creating a quotation against the enquiry. I am accepting the quotation and converting it into a Sales Order. Now I am logging in as Admin. I can see the order and available inventory. I am confirming the order, which reserves the required stock. Finally, I am dispatching the order, and the physical and reserved inventory quantities are updated accordingly.

22. Interview Questions to Prepare

Why use a transaction for inventory reservation?

Because reservation involves database operations that must succeed together. A PostgreSQL transaction with row-level locking prevents simultaneous requests from reserving the same available stock.

Why calculate quotation total on the backend?

The frontend is controlled by the client and cannot be trusted. React can show a preview, but the backend recalculates and validates the amount before saving.

23. Possible Live Verification Change

Example: Add DAMAGED stock.

```
Physical = 100
Reserved = 20
Damaged = 10

Available = 100 - 20 - 10
           = 70
```

Update the database model, backend availability calculation, relevant API, and frontend inventory display.

Another possible change: allow a confirmed Sales Order to be cancelled and correctly release its reserved inventory.

24. Final Mental Model

```
CUSTOMER
↓
ENQUIRY
↓
QUOTATION
↓
ACCEPTED
↓
SALES ORDER
↓
ADMIN CONFIRMS
↓
RESERVE STOCK
↓
DISPATCH
↓
PHYSICAL STOCK ↓
RESERVED STOCK ↓
```

The main evaluation areas are business logic, relational PostgreSQL design, secure backend authorization, data consistency, transactions, tests, and your ability to explain/modify your own code.